

Desarrollo de *ESC* para motores *BLDC* sin sensores de posición, basado en microcontrolador de la gama *STM32*

Francisco Castel
Facultad de Ingeniería - UNCUIYO
Mendoza, Argentina
castel.francisco@uncuyo.edu.ar
<https://github.com/fcastel2002>

Abstract—Este trabajo presenta el desarrollo e implementación de un Controlador Electrónico de Velocidad (ESC) *sensorless* para motores *BLDC/PMSM* utilizando microcontroladores *STM32*. El diseño se centra en un enfoque de bajo costo y complejidad reducida, basado en la detección de cruces por cero (ZCP) de la fuerza contraelectromotriz mediante comparadores analógicos, complementado con un control PI para la regulación de velocidad. La arquitectura propuesta aprovecha periféricos nativos del *STM32F103C8T6* (*timers* avanzados, ADC, *UART* y memoria *Flash*), lo que minimiza la carga computacional y aumenta la robustez del sistema. Se implementó una estrategia de arranque basada en modulación SPWM trifásica en lazo abierto, que asegura una transición estable hacia el régimen *sensorless*. Asimismo, se desarrolló una máquina de estados modular que gestiona los distintos modos de operación (*IDLE*, *RUNNING*, *CLOSED-LOOP*, *CONFIG*, entre otros), mejorando la escalabilidad y confiabilidad del *firmware*. Los ensayos experimentales con motores *BLDC* extraídos de discos rígidos demostraron un rango de operación entre 100 y 5500 rpm, estabilidad térmica en la etapa de potencia basada en L298N, y un control adecuado bajo condiciones de carga predecible. El prototipo constituye una base funcional para aplicaciones donde la simplicidad y la eficiencia prevalecen sobre la necesidad de control dinámico avanzado, tales como ventiladores, sistemas de bombeo y pequeños vehículos eléctricos. Finalmente, se discuten limitaciones del enfoque reactivo y posibles mejoras mediante técnicas predictivas o híbridas, apuntando hacia futuras implementaciones con mayor robustez e integración industrial.

I. INTRODUCCIÓN

Los motores *BLDC-PMSM* (*Brushless DC - Permanent Magnet Synchronous Motors*) representan una tecnología fundamental en aplicaciones que requieren alta eficiencia y control preciso, desde electrodomésticos hasta vehículos eléctricos y sistemas industriales. Su alta densidad de potencia, eficiencia superior (típicamente 85-95%) y mayor vida útil los han convertido en la pieza fundamental durante el desarrollo y evolución de los drones, donde la relación potencia-peso es crítica. Es por esto que la motivación principal de este trabajo es la de entender profundamente el funcionamiento de este tipo de motores y las estrategias y técnicas de control del mismo.

Para lograr este objetivo se propuso la realización de un Controlador Electrónico de Velocidad (ESC) ya que prometía ser un desafío interesante y con cierta complejidad.

La evolución de las técnicas *sensorless* ha sido notable desde que Iizuka et al. [1] propusieron en 1985 un método basado en la detección del *Zero Crossing Point* (ZCP) mediante comparadores externos, estableciendo las bases para futuros desarrollos. Posteriormente, técnicas como las descritas por [2] mejoraron la precisión mediante la medición de *BEMF* durante la ventana de *PWM OFF*. Un avance significativo llegó con Chen [3], quien simplificó el *hardware* necesario aprovechando el voltaje de línea, eliminando la necesidad de cir-

cuitos analógicos adicionales para el cálculo del instante de conmutación.

Actualmente, el Control de Campo Orientado (FOC) representa el estado del arte, ofreciendo control preciso de par y velocidad en un amplio rango operativo. Sin embargo, el FOC requiere algoritmos complejos que demandan mayor capacidad computacional y una medición de corriente con SNR superior a 40dB, lo que incrementa los costos de implementación.

El objetivo del presente trabajo es proporcionar un método sencillo y económico para el control *sensorless* de motores BLDC/PMSM utilizando microcontroladores STM32. El enfoque se basa en la detección de cruces por cero mediante circuitos analógicos simples, complementados con un control PI para regular la velocidad. Esta implementación aprovecha los periféricos del STM32F103C8T6 (ADC, *timer* avanzados, etc) para minimizar la carga computacional, resultando ideal para aplicaciones de ventilación y movimiento de aire donde no se requiere control de posición ni alto par de arranque. El método es aplicable en sistemas de baja y media potencia con velocidades mínimas de operación superiores al 15-20% de la nominal.

Este documento está organizado en secciones que cubren: fundamentos teóricos del control *sensorless*, diseño del hardware basado en STM32, implementación del algoritmo de detección y control, resultados experimentales, y conclusiones con recomendaciones para trabajos futuros.

A. Principio de funcionamiento de motores BLDC - PMSM

Cuando hablamos de motores sin escobillas, existen principalmente dos tipos, los motores con una *BEMF* trapezoidal, categorizados como BLDC, y los que presentan una *BEMF* sinusoidal, como PMSM. Ambos difieren de los motores DC con escobillas en que la conmutación debe realizarse de forma electrónica, y no se realiza por la construcción del mismo. Esta diferencia en la forma de onda de la *BEMF* se debe principalmente a la distribución del bobinado y la configuración del campo magnético del rotor: los BLDC típicamente utilizan bobinados concentrados y magnetización

radial, mientras que los PMSM emplean bobinados distribuidos y, en muchos casos, magnetización sinusoidal.

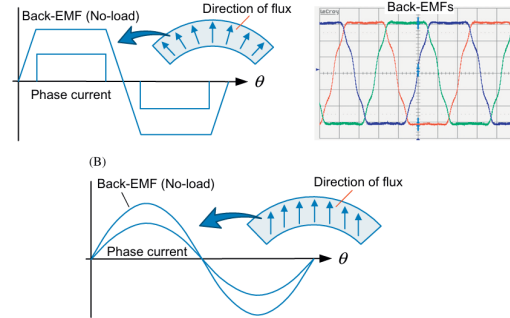


Figure 1. Diferencia sustancial entre BLDC y PMSM

1) *Métodos de detección de cruces por cero*: La detección de los cruces por cero (ZCP) de la *BEMF* constituye el método más implementado para el control *sensorless* de motores BLDC. Este punto ocurre cuando la *BEMF* cambia de polaridad, lo que corresponde a una posición específica del rotor, generalmente desfasada 30° eléctricos del punto óptimo de conmutación.

a) *Detección directa de BEMF*: El método clásico propuesto por Iizuka et al. [1] utiliza comparadores analógicos para detectar el instante en que la *BEMF* cruza el nivel de referencia (generalmente el punto neutro virtual). Esta técnica requiere circuitería de acondicionamiento para:

- Filtrar el ruido de conmutación
- Crear un punto neutro virtual para comparación
- Generar una señal de interrupción al microcontrolador

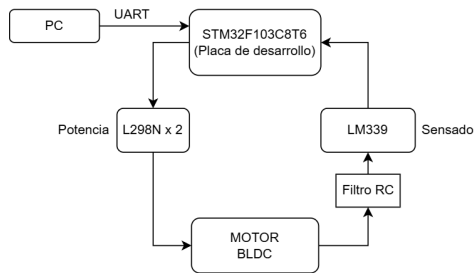
Mas adelante en el presente trabajo, se expresa la elección de este método.

b) *Método de integración de la tercera armónica*: Esta técnica, desarrollada por Moreira en el '96 [2], aprovecha el hecho de que la suma de las tres *BEMF* contiene una tercera armónica significativa. La integración de esta componente proporciona información de posición más robusta frente al ruido, aunque requiere un procesamiento adicional.

c) *Método de terminal único:* Propuesto por Chen et al. [3], este enfoque simplifica la implementación hardware al utilizar solo un terminal para la medición de voltaje de línea. La principal ventaja es la reducción de componentes externos y la posibilidad de implementación completa en microcontroladores con suficientes recursos de ADC.

d) *Detección mediante PWM sincronizado:* Este método, particularmente adecuado para implementaciones en microcontroladores, muestrea la BEMF durante los períodos de apagado del PWM, cuando la corriente de fase es mínima. La técnica requiere una sincronización precisa entre el PWM y la secuencia de muestreo del ADC, factible en microcontroladores avanzados como la familia STM32 que ofrecen disparo sincronizado entre periféricos [4].

II. ESQUEMA TECNOLÓGICO



A. Detalle de módulos

1) *Etapas de potencia: L298N:* La etapa de potencia se implementó utilizando dos módulos basados en el circuito integrado L298N [5], configurados para proporcionar tres medios puentes H necesarios para la conmutación trifásica del motor BLDC. Cada medio puente se controla mediante dos señales: una para el transistor superior y otra para el inferior. Este módulo presenta las siguientes características:

- Voltaje de salida: 5V - 35V
- Voltaje lógico: 5V - 7V
- Corriente máxima por canal: 2A
- Potencia máxima de disipación: 25W

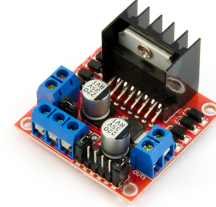


Figure 2. Módulo comercial basado en L298N y sus respectivas etapas de filtrado y feedback.

2) *Motor utilizado:* Para las pruebas se utilizaron dos motores sin escobillas sacados de discos rígidos antiguos, estos fueron una gran elección debido a la alta disponibilidad como chatarra. No fue posible probar el sistema con motores BLDC de alto rendimiento como los presentes en drones comerciales. Así como estos motores son sencillos de conseguir, también es muy difícil de encontrar alguna referencia de ellos en internet ya que la mayoría de códigos de referencia y/o números, eran más bien códigos internos y no existen como tales, por lo que no se consiguieron datos del motor como tal. Aunque un parámetro que sí era relevante para saber si el algoritmo estaba funcionando de una forma adecuada era la velocidad máxima, y la de los discos rígidos antiguos solía ser de no más de 5400 rpm, a diferencia de los actuales que llegan a las 7200 rpm.

Para que se tenga una noción de la antigüedad de estos motores, los discos duros de los que fueron extraídos usaban motores paso a paso para mover la aguja de lectura.

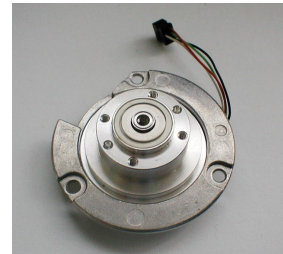


Figure 3. Motor extraído de un disco rígido antiguo

3) *Circuito externo de detección:* Para la detección de cruces por cero de la fuerza contraelectromotriz (BEMF), se implementó el método propuesto por Iizuka [1], que utiliza comparadores analógicos para detectar los instantes en que la BEMF de la fase flotante cruza el nivel del punto neutro virtual. La fig:zero_crossing_muestra el circuito implementado.

En esta configuración, las resistencias R4-R6 (10 k Ω) crean un punto neutro virtual, lo que flexibiliza el tipo de bobinado de motor, ya que sea Estrella o Delta (acceso al neutro real o no) se utilizan solo las terminales de las tres fases. Luego las resistencias R1-R3 (10k Ω) y capacitores C1, C2 y C3 (47 nF) forman una etapa de filtrado para reducir el ruido de conmutación y propio del motor. Los comparadores del LM339 [6] detectan el momento exacto del cruce por cero (aun así se dota de histéresis mediante resistencias de 100 k Ω), generando flancos que son capturados por los canales de Input Capture (IC_CH1, IC_CH2 e IC_CH3) del temporizador del microcontrolador.

Se seleccionó el LM339 debido a su salida de colector abierto, que permite una interfaz segura entre los 12V del circuito de potencia y los 3.3V del STM32F103C8T6, sin necesidad de circuitos adicionales de adaptación de nivel.

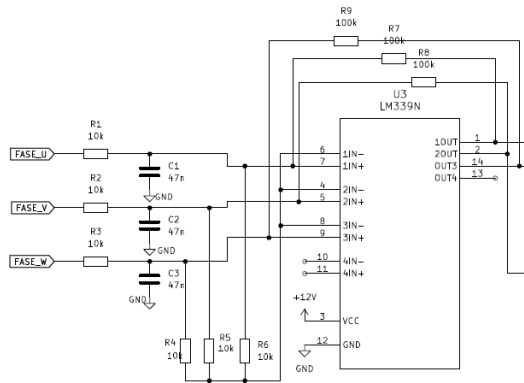


Figure 4. Circuito implementado

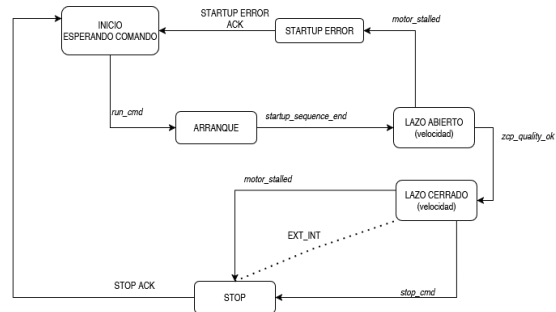
Se decide la utilización de un comparador de

colector abierto como el LM339 [6] debido a su versatilidad al momento de interactuar con dispositivos de voltajes sumamente distintos al de alimentación, como en este caso, 12V contra los 3.3 V del STM32F103C8T6.

III. FUNCIONAMIENTO GENERAL

El objetivo principal del controlador es el de lograr la consigna de velocidad en el motor, para ello debe recibir los cruces por cero y a partir de ellos realizar las conmutaciones correspondientes. El funcionamiento del firmware en sí es sencillo, lo que hace es:

- 1) Rutina de arranque.
- 2) Realizar la conmutación cuando se recibe el flanco correspondiente desde el circuito comparador.
- 3) Medir el periodo entre dos flancos de subidas de una misma fase.
- 4) Cada cierto tiempo (regido por la velocidad máxima) actualizar la salida PWM mediante un control PI.



Pero esto no es suficiente para una conmutación robusta, fue sumamente necesario incluir un algoritmo sencillo de detección de la consistencia de la señal del cruce por cero. Esto es crucial para sobre todo el funcionamiento en lazo cerrado.

IV. PROGRAMACIÓN

A. Configuración de periféricos del microcontrolador

El microcontrolador STM32F103C8T6 se configuró para realizar tanto la detección de cruces por

cero como la generación de señales de control para los puentes H:

- **timer 2:** Configurado con tres canales en modo Input Capture para detectar los flancos provenientes de los comparadores. Este enfoque, en lugar de utilizar interrupciones externas (EXTI), permite medir directamente el período entre cruces por cero, facilitando el cálculo de la velocidad del motor y la temporización precisa de la conmutación.
- **timer 1:** Utilizado como generador PWM para controlar los transistores superiores de los medios puentes. Se configuró para operar en un rango de frecuencias entre 10 y 24 kHz (no fue necesaria la implementación de tiempo muerto dado que el retraso propio de la conmutación proporciona una especie de dead time).
- **UART1:** Periférico UART configurado a 115200 de baudrate utilizado en modo interrupción.
- **CRC:** Periférico utilizado para la verificación de integridad de datos. Este módulo permite realizar cálculos rápidos de verificación, contribuyendo a la robustez en la transmisión y almacenamiento de parámetros críticos, se utiliza para la persistencia de parámetros en FLASH.
- **Memoria Flash:** Utilizada para almacenamiento no volátil de parámetros de configuración y calibración del ESC. La memoria Flash permite conservar configuraciones esenciales entre reinicios o ciclos de alimentación. Permite al operario configurar su ESC como desee.

1) *Señales PWM:* La técnica PWM seleccionada corresponde a "PWM technique 1" en la fig:pwm-tech, también conocida como PWM unipolar. En esta configuración, durante cada sector de 60° eléctricos, una fase se conecta al positivo mediante PWM, otra fase se conecta al negativo de forma permanente, y la tercera fase permanece flotante (desconectada). Se eligió esta técnica ya que la implementación de las demás técnicas, según Lai et al en [7], pueden cobrar sentido si se utiliza un método de sensado directo de la BEMF mediante ADC.

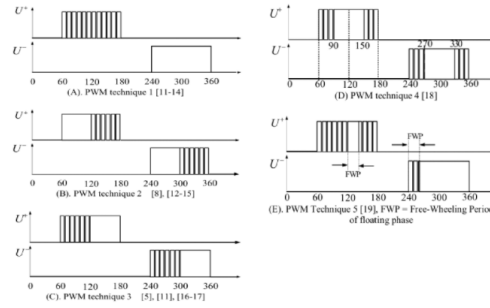


Figure 5. Resumen de técnicas PWM para motores BLDCM. [7]

B. Estrategia de arranque

El arranque representa uno de los mayores desafíos en el control *sensorless* de motores BLDC, ya que a velocidad cero o muy baja no se genera suficiente BEMF para detectar la posición del rotor. En este trabajo se evaluaron diferentes estrategias de arranque antes de seleccionar la implementación final.

1) *Alternativas consideradas:* La técnica *Align & Go*, ampliamente documentada en la literatura, fue inicialmente considerada debido a su simplicidad de implementación. Sin embargo, esta técnica presenta una desventaja significativa: la posibilidad de que el rotor inicie su movimiento en dirección opuesta a la deseada. Si bien existen algoritmos para detectar y corregir esta situación, su implementación requiere una detección casi instantánea del sentido de rotación y una rápida inversión de la secuencia de conmutación, aumentando considerablemente la complejidad del sistema.

Otra alternativa analizada fue el método IPD (*Initial Position Detection*), implementado por Meghana et al en [8], que permite un arranque desde velocidad cero mediante la detección de la posición inicial del rotor. A pesar de ser el método más utilizado en aplicaciones comerciales de control *sensorless*, la complejidad de sus algoritmos y los requisitos de sincronización precisa entre periféricos lo situaron fuera del alcance de este trabajo.

2) *Solución implementada: SPWM en la etapa de arranque:* La estrategia de arranque adoptada se basa en un generador senoidal de tres fases

mediante modulación por ancho de pulso senoidal (SPWM), implementado completamente en lazo abierto. Esta técnica consiste en:

- 1) Precálculo de tres tablas senoidales desfasadas 120° eléctricos entre sí (una por fase), ajustadas al formato Q15 y escaladas para facilitar su modulación en tiempo real.
- 2) Generación periódica de valores PWM actualizados a partir de dichas tablas, mediante un temporizador auxiliar que actualiza el índice de fase a intervalos controlados.
- 3) Aplicación de estas señales al puente inversor trifásico del motor, con habilitación de las tres fases y control de flotación durante toda la etapa de arranque.
- 4) Incremento controlado de la frecuencia efectiva del campo rotante a través de la reducción progresiva del ARR (Auto-Reload Register) del temporizador auxiliar.
- 5) Aumento gradual del valor de modulación (`mod_q15`), que controla la amplitud de la señal aplicada, desde un valor inicial bajo ($\approx 20\%$) hasta una amplitud estable cercana al 100%.
- 6) Finalización automática del arranque tras un número fijo de iteraciones (`STARTUP_ITERATIONS`), momento en el cual se realiza la transición controlada al modo de conmutación trapezoidal *sensorless*.

La lógica de control se implementa utilizando el temporizador TIM4 del STM32F103C8T6 para las actualizaciones periódicas de PWM, mientras que el generador de PWM (TIM1) continua modulando la señal SPWM. Durante el arranque, el sistema no requiere retroalimentación alguna del motor: no se evalúa la *BEMF*, ni se mide corriente, ni se utiliza información de posición.

La transición al modo de operación *sensorless* se realiza de forma determinista, tras agotar el tiempo definido para el arranque. En ese instante, se detiene la modulación SPWM, se deshabilitan momentáneamente las salidas, y se reinicializa el sistema para comenzar con la conmutación trapezoidal convencional, habilitando el sistema de detección de cruces por cero.

3) *Ventajas y limitaciones*: Esta aproximación presenta varias ventajas relevantes para la aplicación:

- Permite un arranque suave y robusto, incluso en ausencia total de información de posición o velocidad.
- Es insensible a la posición inicial del rotor, gracias a la naturaleza continua del campo magnético rotante generado.
- Su implementación es simple y determinista: no requiere ajustes dinámicos durante el arranque, ni evaluación de condiciones de éxito o fallo.

Como limitación principal, el sistema no es capaz de detectar si el motor ha sincronizado efectivamente durante la etapa de arranque. El método asume que, al finalizar el tiempo definido, el rotor habrá sido capaz de seguir el campo magnético generado. Esto representa un riesgo en condiciones de carga elevada o con el rotor bloqueado, ya que la transición podría realizarse sin que el motor haya adquirido velocidad efectiva.

Cabe destacar, sin embargo, que si bien durante el arranque no se evalúa la consistencia de los cruces por cero debido a que la señal de *BEMF* queda enmascarada por la modulación senoidal (haciendo que cualquier cruce detectado en esta etapa sea irrelevante o espurio), el sistema implementa una verificación posterior. Tras completarse el tiempo de arranque y al entrar al modo *sensorless*, si los cruces por cero detectados no presentan la coherencia esperada, se considera fallida la sincronización y se vuelve automáticamente a ejecutar la maniobra de arranque desde cero. Esta estrategia añade una capa de robustez frente a condiciones anómalas sin complejizar la lógica de control durante el arranque propiamente dicho.

C. Conmutación

Para conmutación de las fases, se optan dos estrategias cruciales para no perder la sincronización tan buscada desde la etapa de arranque entre los cruces por cero, la posición del rotor y los pasos de la conmutación.

1) *Fase flotante como variable global*: Un problema que surgía al momento de utilizar el método

de Iizuka es que si no se asignaba un orden de prioridad a las interrupciones era factible que uno de los canales detectara una segunda interrupción antes de que el canal de la fase que nos interesa (la fase flotante) detectase su flanco, provocando esto una conmutación a destiempo. Por esto se deciden crear tres variables booleanas (`float_U`; `float_V` y `float_W`) que se encargan de informar al callback de interrupciones de cual será la próxima fase flotante, es decir, el próximo canal que debe atender. Por ejemplo, para el paso de conmutación que energiza las fases U y V (+U -V) el trio de variables tomaría los siguientes valores:

- `float_U` = false
- `float_V` = false
- `float_W` = true

Agregando así un condicional adicional al callback de interrupción.

2) Alternado de polaridad del input capture:

Un factor indispensable a tener en cuenta durante el desarrollo de la conmutación fue el de la necesidad de que los canales del input capture correspondientes a cada fase pudiesen captar tanto flancos de bajada como de subida, lamentablemente esto no esta permitido para el canal 3 del *timer 1* (aun siendo este un *timer avanzado*) por lo que este inconveniente se soluciono cambiando la polaridad de cada canal de forma alternada, es decir que, por ejemplo, el canal que corresponde a la fase U, para la primer posicion dentro de la secuencia se establece con polaridad de flanco de bajada, y luego para el proximo paso donde la fase U es flotante, se establece la polaridad de flanco de subida, esto se repetirá para cada canal. Esta implementación no solo solucionó las incapacidades del *timer 1*, si no que propuso más robustez al sistema previniendolo de falsos cruces por cero dados por posible ruido, así, es imposible que un flanco de bajada sea detectado previo a otro flanco de bajada del mismo canal.

D. Sistema de máquina de estados

La máquina de estados constituye la columna vertebral del controlador, gestionando las transiciones entre los diferentes modos de operación.

Table I
SECUENCIA DE CONMUTACIÓN DE 6 PASOS PARA MOTOR BLDC

2*Paso	Enable			PWM			2*Polaridad	Flotantes		
	U	V	W	U	V	W		U	V	W
POS_UV	1	1	0	PWM	0	-	Fall(CH1)	No	No	Sí
POS_UW	1	0	1	PWM	-	0	Rise(CH3)	No	Sí	No
POS_VW	0	1	1	-	PWM	0	Fall(CH2)	Sí	No	No
POS_VU	1	1	0	0	PWM	-	Rise(CH1)	No	No	Sí
POS_WU	1	0	1	0	-	PWM	Fall(CH3)	No	Sí	No
POS_WV	0	1	1	-	0	PWM	Rise(CH2)	Sí	No	No

La función `handleState()` implementa un patrón de diseño basado en estados que permite:

- Separar claramente la lógica para cada modo de operación
- Gestionar transiciones controladas entre estados
- Mantener activas solo las interrupciones necesarias en cada estado

V. RESUMEN DEL FUNCIONAMIENTO DE LA MÁQUINA DE ESTADOS

El módulo de la máquina de estados ('state_machine.c') implementa el control centralizado del ESC mediante una estructura clara y robusta basada en estados. Las funciones y características principales son:

- **handleState():** Gestiona la transición entre estados, procesando eventos internos y externos, asegurando una ejecución ordenada y segura del ESC.
- **Estados clave:**
 - **IDLE:** Estado inicial o de espera sin actividad del motor.
 - **PROCESS_COMMAND:** Procesa comandos recibidos desde el módulo de comunicación.
 - **FOC_STARTUP:** Maneja la secuencia inicial de arranque.
 - **RUNNING:** Estado operativo básico, gestionando velocidad abierta.
 - **CLOSEDLOOP:** Control operativo avanzado con ajuste automático por realimentación.
 - **CONFIG:** Realiza configuraciones recibidas por comandos externos.

- **READY:** Preparación intermedia para el control de lazo cerrado.
- **STOPPED:** Estado de detención del motor.
- **FINISH:** Finaliza operaciones y retorna a IDLE.
- **HARD_ERROR:** Manejo de errores críticos que requieren intervención.

La máquina de estados garantiza un control robusto, previniendo errores críticos mediante transiciones claras y supervisadas, y actúa como núcleo central que coordina la interacción y comunicación efectiva entre todos los módulos del sistema. Su estructura es fácilmente escalable, facilitando la incorporación de nuevos estados y eventos según sea necesario.

A. Funciones mas importantes del control

- **commutation()**
Controla la secuencia de activación/desactivación de las fases U, V, y W del motor brushless según el paso actual de conmutación.
Establece las polaridades adecuadas y asigna valores PWM específicos a cada fase según la posición del rotor.
- **zero_crossing()**
Gestiona el control del motor mediante la detección de cruces por cero (*Zero Crossing Points* - ZCP).
Realiza la conmutación del motor en cada cruce por cero detectado en cualquiera de las fases.
Actualiza la velocidad usando mediciones trifase para calcular un promedio por consenso entre las fases (método robusto).
- **pi_control()**
Implementa un controlador Proporcional-Integral (PI) para ajustar el PWM del motor según un setpoint de velocidad.
Utiliza parámetros configurables (KP, KI) para el control fino de la velocidad y limita la salida PWM dentro de rangos seguros.
- **check_motor_status()**
Monitorea periódicamente el estado operativo del motor.

Detecta condiciones de bloqueo (stall) mediante la comprobación de tiempo desde el último cruce por cero válido. Establece una bandera (*motor_stalled*) para indicar situaciones críticas.

- **stop_motor()**
Maneja el procedimiento para detener el motor en dos modos:
Detención gradual (modo normal).
Detención inmediata de emergencia (modo crítico), con cambio de dirección temporal para frenado rápido.
- **calculate_consensus_speed()**
Calcula la velocidad promedio por consenso utilizando mediciones consistentes de múltiples fases, lo que mejora significativamente la robustez ante ruidos o fallos ocasionales en la detección de cruces por cero.

B. Sistema de comunicación

El controlador implementa un protocolo de comunicación basado en comandos textuales a través de UART, permitiendo la configuración y monitorización del sistema.

1) *Resumen del funcionamiento de la comunicación:* El módulo de comunicación ('comm.c') implementa la interfaz UART para interacción externa con el ESC. Las funciones más importantes son:

- **commInit():** Inicializa el buffer de recepción y comienza la recepción por interrupción usando UART.
- **processUartData():** Lee y procesa bytes recibidos del buffer circular. Detecta comandos completos al recibir caracteres finales como \r o \n, almacenándolos para su ejecución.
- **processCommand():** Analiza comandos recibidos, los valida sintácticamente y determina la acción a realizar, pudiendo ejecutar acciones como RUN, STOP, EMERGENCY_STOP o ajustes específicos como la frecuencia PWM.
- **executeCommand():** Ejecuta la acción indicada en el comando analizado (por ejemplo, configurar la frecuencia PWM o iniciar la

maniobra de arranque FOC), devolviendo un estado de confirmación.

- **transmitirUART():** Envía respuestas por UART usando transmisiones no bloqueantes por interrupción, informando estados o errores al usuario externo.

Este módulo asegura una comunicación robusta y eficiente con la interfaz externa, permitiendo el control dinámico del ESC mediante comandos estructurados. Además, su diseño es altamente escalable, ya que permite fácilmente la incorporación de nuevas acciones y parámetros para extender o adaptar el conjunto de comandos disponibles.

VI. ETAPAS DE MONTAJE Y ENSAYOS REALIZADOS

En esta sección se describen las principales etapas del montaje del sistema ESC y los ensayos efectuados para verificar su funcionamiento.

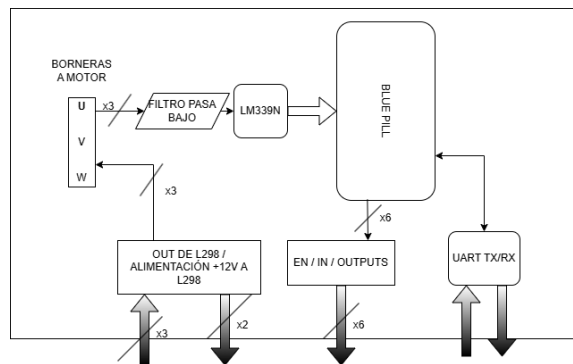


Figure 6. Esquemático conceptual de conexiones y partes del prototipo

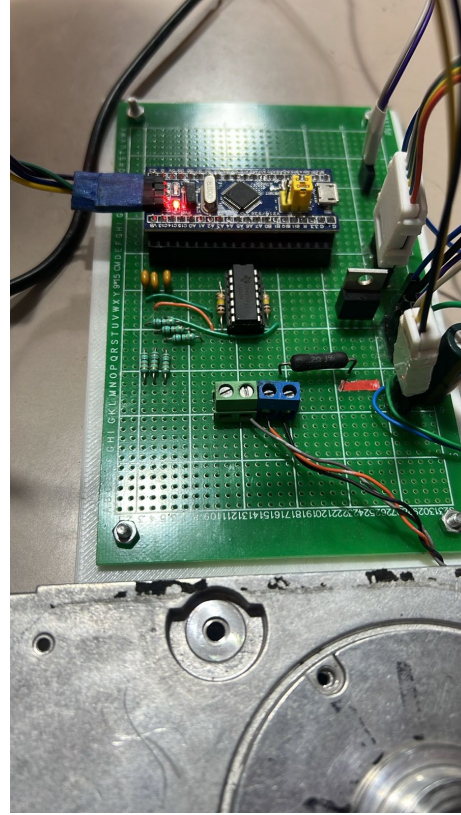


Figure 7. Montaje del circuito lógico en la placa de prototipo.

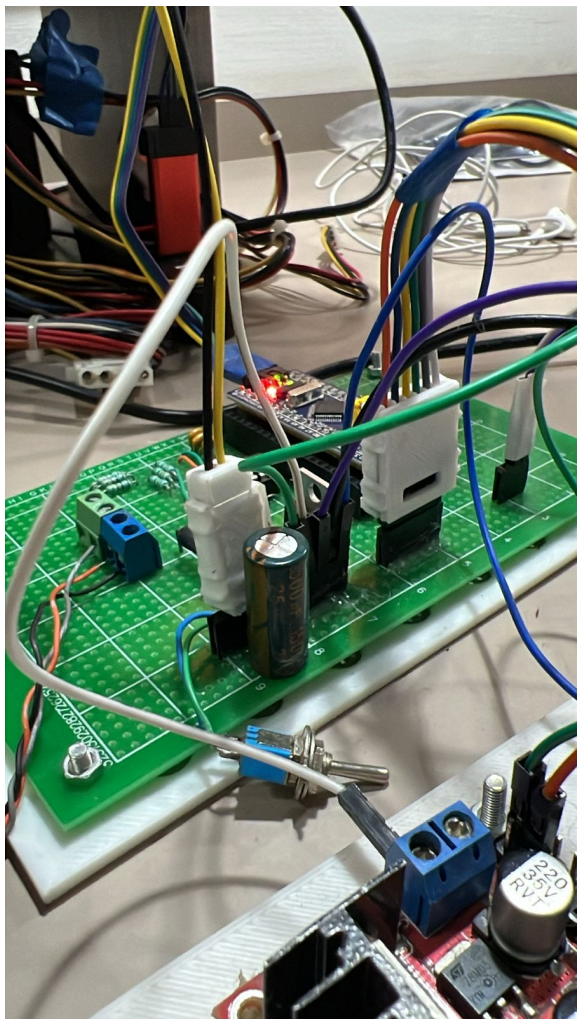


Figure 8. Otra vista del circuito lógico.

En la imagen 8 y 7 se puede ver la protección a polaridad inversa mediante un Diodo Schottky y un capacitor de gran capacidad (1500 uF) para protección contra picos de corriente y estabilidad del MCU.

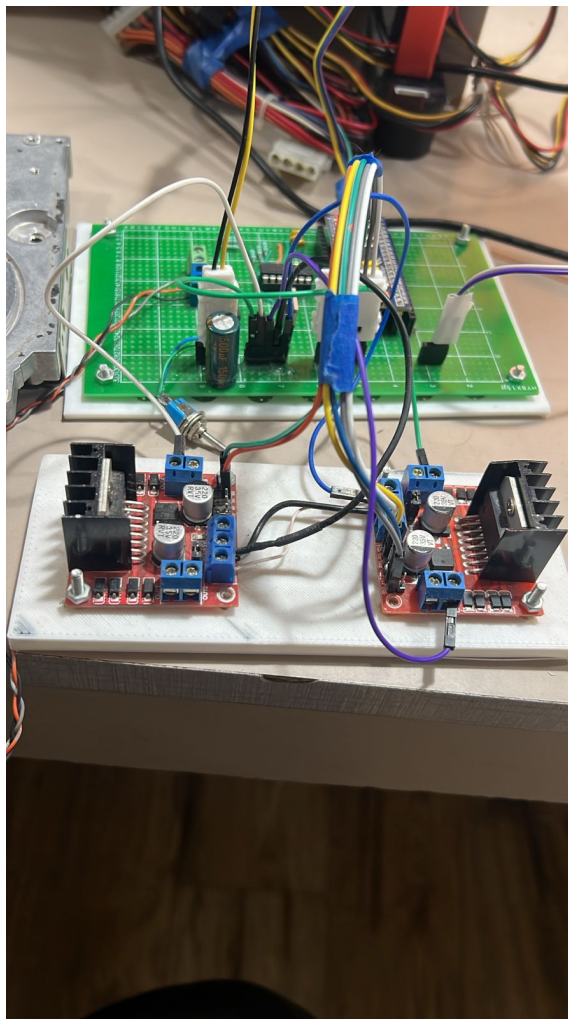


Figure 9. Etapa de potencia (2x L298N) y circuito lógico.

Debe entenderse que un prototipo final no tendría la etapa de potencia separada en módulos como este caso, sino mas bien en una misma placa, la etapa de potencia y la etapa de control.

VII. RESULTADOS Y ESPECIFICACIONES FINALES

Se presentan resultados cualitativos y cuantitativos del proyecto, obtenidos a partir de distintas pruebas.

- Rango de operación: Sin ninguna carga, el motor, el cual nominalmente está diseñado en

5400 rpm, según las lecturas del osciloscopio muestra una velocidad máxima de 5500 rpm, y una velocidad mínima de 100 rpm.

- La etapa de potencia no presenta disipación térmica notable, con los disipadores presentes en los módulos L298, en términos generales, por lo mencionado, la temperatura de los mosfets no supera los 40°C.
- Limitante principal y próximo paso a mejorar: El hecho de que el sistema sea reactivo, y no predictivo, produce un comportamiento esperable, que no se puede confiar ciegamente en las lecturas de cruces por cero, por lo que es muy sensible a una desincronización. Se mejoraría este aspecto logrando un método predictivo, pero se soluciona por completo el enmascaramiento de la señal del cruce por cero tomando un enfoque de lectura de corrientes, sin entrar en algoritmo FOC.
- Control pobre en términos generales: No es posible realizar cambios bruscos de velocidad, ya que estos generan picos de corriente en la señal que distorsionan por completo la lectura de los cruces por cero, y debido a la naturaleza reactiva de estos, el control pierde sentido. Una solución muy interesante e investigada podría ser la implementación de control difuso para estas consignas de velocidad bruscas.

VIII. CONCLUSIONES. ENSAYO DE INGENIERÍA DE PRODUCTO

Aplicabilidad del sistema

Si bien el ESC no está diseñado para aplicaciones que requieran precisión dinámica elevada o control activo de par, resulta especialmente adecuado para sistemas donde el control de velocidad sea el requerimiento principal y las condiciones de carga se mantengan dentro de márgenes predecibles.

Entre las aplicaciones posibles se incluyen:

- Sistemas de bombeo de agua o fluidos industriales.
- Ventiladores centrífugos o axiales de media y baja potencia.
- Propulsión de pequeños drones o vehículos autónomos.

- Sistemas de refrigeración, circulación y climatización.

Análisis detallado: bomba centrífuga de agua

Para validar más rigurosamente la viabilidad del controlador en una aplicación real, se analiza su desempeño esperado en el contexto del accionamiento de una bomba centrífuga.

Este tipo de carga presenta un perfil de par creciente con la velocidad, típicamente proporcional al cuadrado de la misma ($T \propto \omega^2$). Además, las variaciones de carga durante el funcionamiento son suaves, lo que reduce las exigencias dinámicas del sistema de control. Asimismo, no se requiere inversión del sentido de giro ni regulación precisa del par instantáneo.

El ESC desarrollado cumple con los requerimientos básicos de esta aplicación:

- **Arranque suave:** la rampa de frecuencia en modo SPWM permite una aceleración gradual, limitando los picos de corriente sin necesidad de sensado.
- **Control de velocidad estable:** tras el arranque, el sistema opera en lazo abierto con conmutación sincronizada a los cruces por cero, ofreciendo estabilidad suficiente en aplicaciones con dinámica lenta, como una bomba hidráulica.
- **Protección frente a bloqueo:** el firmware implementa un mecanismo de detección de pérdida de sincronía mediante el monitoreo de cruces por cero. Ante inconsistencias sostenidas, el sistema reinicia la secuencia de arranque o detiene la conmutación, funcionando como una protección frente a bloqueo mecánico.
- **Compatibilidad con el entorno operativo:** la topología sin sensores y el bajo número de componentes electrónicos favorecen la integración del controlador en entornos húmedos o de difícil acceso, donde la robustez y el bajo mantenimiento son prioridades.

Selección final de componentes y esquema de protección

1. Motor de aplicación

- Tipo: Brushless trifásico *sensorless*, 12–24 V.
- Ejemplo comercial: *Turnigy D2830/11 1000 KV* (100 W continuo @ 12–14.8 V, $I_{\max} \approx 10$ A).

2. Etapa de potencia

- **MOSFETs:** $6 \times$ *IRLZ44N* (55 V, 47 A, $R_{DS(on)} \leq 22 \text{ m}\Omega$). Montados sobre disipador común de aluminio ($R_{\theta SA} \leq 2^\circ\text{C/W}$).
- **Driver:** $3 \times$ *IR2101S* (medio-puente alto/bajo, con bootstrap). $V_{HB} \leq 600$ V – sobredimensionado para 24 V. Capacidad de corriente de puerta: ± 2 A (tiempo típico de carga ≈ 150 ns para $Q_g \approx 100$ nC).
- Resistores de puerta: $R_g = 10\text{--}12 \Omega$ para amortiguación de *ringing*.

3. Controlador principal

- *STM32F103C8T6* (Cortex-M3, 72 MHz), validado en el prototipo.
- Oscilador externo de 8 MHz tipo HC-49S para mayor precisión temporal (PWM y captura).

4. Fuente y filtro de entrada (24 V DC)

- Fuente conmutada industrial 24 V / 6 A (*Mean Well LRS-150-24* o equivalente).

5. Protección y aislamiento de señales

- Para proteger el microcontrolador y reducir los efectos de perturbaciones eléctricas generadas durante la conmutación o provenientes de la red de alimentación, se recomienda incorporar aislamiento entre la etapa de control y la etapa de potencia. Esto puede lograrse mediante **optoacopladores** rápidos, especialmente en las señales que activan los drivers de compuerta.
- La inclusión de filtros pasivos (como redes RC o snubbers) en las fases de salida puede ayudar a atenuar transitorios generados durante los cambios de estado de los MOSFETs. Estos elementos también pueden reducir la emisión de interferencias electromagnéticas hacia otros dispositivos sensibles.
- En la entrada de alimentación, es aconsejable emplear un filtro de tipo *Pi* o similar,

con inductores y capacitores adecuados a la corriente del sistema, para minimizar los efectos de ruido conducido, tanto emitido como recibido.

- Adicionalmente, la inclusión de un sensor de temperatura sobre el disipador de los MOSFETs permitiría implementar una lógica de protección térmica, deteniendo el sistema ante un sobrecalentamiento prolongado.

En conclusión, el prototipo desarrollado constituye una base funcional para un ESC *sensorless* de bajo costo. Sin embargo, su viabilidad industrial requiere una reingeniería de la etapa de potencia, mejoras en el firmware y una selección cuidadosa de componentes que permitan aumentar su eficiencia, confiabilidad y adaptabilidad a entornos reales. Se provee del link al repositorio. <https://github.com/fcastel2002/esc-bldc-sensorless-stm32>

REFERENCES

- [1] K. Iizuka, H. Uzuhashi, M. Kano, T. Endo, and K. Mohri, "Microcomputer control for sensorless brushless motor," *IEEE Transactions on Industry Applications*, vol. IA-21, no. 3, p. 595–601, May 1985. [Online]. Available: <http://dx.doi.org/10.1109/TIA.1985.349715>
- [2] J. Moreira, "Indirect sensing for rotor flux position of permanent magnet ac motors operating over a wide speed range," *IEEE Transactions on Industry Applications*, vol. 32, no. 6, p. 1394–1401, 1996. [Online]. Available: <http://dx.doi.org/10.1109/28.556643>
- [3] S.-H. Chen, T.-H. Liu, and F.-C. Hsiao, "Sensorless control of the bldc motors using a kalman filter," in *2007 International Conference on Machine Learning and Cybernetics*, vol. 4, 2007, pp. 1934–1939.
- [4] STMicroelectronics, *RM0008 Reference Manual: STM32F101xx, STM32F102xx, STM32F103xx, STM32F105xx and STM32F107xx advanced ARM-based 32-bit MCUs*, STMicroelectronics, Geneva, Switzerland, August 2021, rev 21. [Online]. Available: https://www.st.com/resource/en/reference_manual/rm0008-stm32f101xx-stm32f102xx-stm32f103xx-stm32f105xx-and-stm32f107xx-advanced-armbased-32bit-mcus-stmicroelectronics.pdf
- [5] STMicroelectronics, *L298N Dual Full-Bridge Driver*, 2023.
- [6] Texas Instruments, *LM339, LM239, LM139, LM2901 Quad Differential Comparators*, Texas Instruments Incorporated, July 2022, sLCS006AA - Revised July 2022. [Online]. Available: <https://www.ti.com/lit/ds/symlink/lm339-n.pdf>
- [7] Y.-S. Lai and Y.-K. Lin, "A unified approach to zero-crossing point detection of back emf for brushless dc motor drives without current and hall sensors," *IEEE Transactions on Power Electronics*, vol. 26, no. 6, pp. 1704–1713, 2011.
- [8] R. Meghana and R. R. Singh, "Sensorless start-up control for bldc motor using initial position detection technique," in *2020 IEEE International Conference on Power Electronics, Smart Grid and Renewable Energy (PESGRE2020)*. IEEE, Jan. 2020. [Online]. Available: <http://dx.doi.org/10.1109/PESGRE45664.2020.9070409>